

GISELE → Plataforma COPERNICUS (MPSP/INPE) — Análise de Aderência e Passos de Adequação

Documento de trabalho. Base: Resumo_Plataforma_COPERNICUS.pdf (4 cards: Visualizador, Editor Vetorial, Gerador de Iframes, Portal de Importação) × estado atual do GISELE (HANDOVER_GISELE.md, v2.x). Gerado em 2026-06-02.

1. O que o documento COPERNICUS especifica

A plataforma COPERNICUS é **cloud-native** e abandona o servidor de mapas monolítico (GeoServer/WMS/WFS-T) em favor de um fluxo direto. O padrão comum aos quatro módulos é:

- **FastAPI** como back-end e *gateway* central (camada de serviço única);
- **MinIO (S3)** guardando rasters em **COG**; **PostGIS** guardando vetores;
- **TiTiler** servindo tiles raster a partir dos COG; **endpoint MVT customizado** (FastAPI) servindo vetores do PostGIS, com controle de acesso por SQL dinâmico a partir das *claims* do JWT;
- catálogo **STAC / pgSTAC** (busca espaço-temporal via CQL2-JSON);
- segurança por **JWT do Login Único gov.br**, filtros por *role*/departamento, **URLs assinadas** com TTL curto, “a UI nunca é fonte de verdade”;
- **Redis** (cache, locks, blocklist) e **Celery** (conversões pesadas em background);
- **CRS sem reprojeção destrutiva** no armazenamento (reprojeção *on-the-fly* na exibição e na escrita);
- **cadeia de custódia auditável, two-person rule** e *compliance* com a **Meta 1 do MPSP**.

Os quatro módulos: **Card 1 - Visualizador** (React + MapLibre, TiTiler/COG, MVT, STAC, XAI, *time-slider*, *swipe*); **Card 2 - Editor Vetorial** (“QGIS leve” via REST+PostGIS, *snapping*, lock otimista, tabelas temporais); **Card 3 - Gerador de Iframes** (URLs assinadas com bbox/layers/exp/jti, blocklist Redis, marca d’água, *kill switch*); **Card 4 - Portal de Importação** (upload TUS, quarentena, Celery+GDAL→COG, *two-person*, anti-duplicação).

2. O que o GISELE é hoje

O GISELE é, na prática, um **visualizador geoespacial client-side de alto nível**, não uma plataforma de servidor:

- **Front-end monolítico**: HTML único (~1,6 MB) + JavaScript em IIFE (sem framework), empacotado em app **Electron**. Renderização por **canvas próprio** (SisMOM_Map, projeção Mercator/PlateCarree, tiles XYZ Esri/OSM/Topo, pan/zoom, anotações).
- **Raster no cliente**: **decodificador GeoTIFF próprio** (sem lib externa) + sistema de paletas + contornos (marching squares) + modo PNG/GIF a partir do FTP do CPTEC. Não há COG, MinIO nem TiTiler.
- **Vetor no cliente**: parsers próprios de GeoJSON/KML/Shapefile, desenhados no canvas. Não há PostGIS nem MVT.
- **Fontes de dados**: FTP do CPTEC (GeoTIFF/PNG), INPE Queimadas (KML), aviationweather.gov (METAR/JSON), arquivos locais, e a recém-criada base de **pontos do usuário** + lista de estações em arquivo (miscelaneas/estacoes_metar.json).
- **“Helper” Python opcional** (FastAPI + rasterio + httpx) embarcado no Electron como *subprocess* — só acelera extração temporal/perfil/calculadora; **não** é um gateway, catálogo ou DB.
- **Persistência**: localStorage + arquivo de config JSON em %APPDATA% (Electron). **Sem autenticação, sem multiusuário, sem auditoria, sem object storage, sem fila assíncrona.**

Conclusão do diagnóstico: a sobreposição real entre os dois é o **papel de Visualizador (Card 1)**. Os Cards 2, 3 e 4 são essencialmente *back-ends* novos que o GISELE não possui. A maior parte da “adequação” não é refatorar o GISELE — é **construir a plataforma de servidor** que a especificação exige e **reposicionar o GISELE (ou sua UX) como o front-end do Card 1**.

3. Aderência por módulo

Legenda: ✓ atende · ◐ parcial / adaptável · ✗ ausente (a construir).

Card 1 — Visualizador

Requisito da spec	GISELE hoje	Status	Ação de adequação
Stack React + MapLibre GL	Vanilla JS + canvas próprio	🕒	Decisão estratégica (§4): migrar o front para MapLibre ou manter o canvas consumindo as mesmas APIs.
Raster via TiTiler + COG (MinIO)	Decodifica GeoTIFF no cliente	🕒	Mover o raster para COG no MinIO + TiTiler; o visualizador passa a consumir tiles (o GISELE já desenha tiles XYZ — reusável).
Valor do pixel via /point	HUD de valor sob o cursor (cliente)	🕒	Trocar a leitura local pelo endpoint /point do TiTiler.
Vetores via MVT (PostGIS)	GeoJSON/KML parseados no cliente	🕒	Endpoint MVT no FastAPI; MapLibre renderiza MVT nativamente.
Catálogo STAC / busca CQL2	Config manual de modelos/bases	✗	Integrar stac-fastapi/pgSTAC; TOC vira navegação STAC.
Segurança JWT gov.br + roles	Sem auth	✗	Gateway valida JWT; vetor filtrado por WHERE de role; raster por asset STAC com URL assinada.
Time-slider temporal	Animação temporal + série temporal	✓	Reaproveitar; ligar ao eixo temporal do STAC.
Comparação por <i>swipe</i>	Multipainel lado a lado (1-4)	🕒	Adicionar cortina/ <i>swipe</i> (o multipainel já cobre boa parte).
Painel de IA Explicável (XAI/SHAP/LIME)	—	✗	Novo painel que consome artefactos XAI (privilegio VIEW_XAI).
Pendências da spec: identificação de feições	Clique → popup de atributos	✓	Já implementado (misc/monit/pontos).
↳ medição de distâncias/áreas	Régua/área/retângulo/perfil	✓	Já implementado.
↳ opacidade granular	Slider de opacidade por camada	✓	Já implementado.
↳ exportação visual PNG	Exportar PNG + vídeo MP4/WebM	✓	Já implementado (PDF é o que falta).
↳ exportação PDF	—	🕒	Falta compositor PDF (mapa+legenda+escala).
↳ <i>spatial bookmarks</i>	—	✗	Novo (salvar viewport nomeado).
↳ filtros espaciais <i>on-the-fly</i>	Recorte por polígono no export	🕒	Estender para filtro de exibição.

Observação importante: várias “Pendências” que a spec lista para o Card 1 **o GISELE já resolve** (identificação de feições, medição, opacidade, exportação de imagem/vídeo). A maturidade de *interface* do GISELE é um ativo a preservar — o esforço do Card 1 concentra-se na **troca do motor de dados** (TiTiler/MVT/STAC) e na **segurança**, não nas ferramentas de usuário.

Card 2 — Editor Vetorial (“QGIS leve”)

Requisito	GISELE hoje	Status	Ação
Desenho de geometrias (vértices, split, merge)	Desenho de polígono/linha/área + polígonos do usuário	🕒	UI de desenho reusável; falta split/merge servidor.
Transação RESTful (POST/PUT só da geometria)	Tudo em memória/localStorage	✗	API REST FastAPI + PostGIS.
Reprojeção ST_Transform no servidor	Render preserva origem (cliente)	✗	Reprojetar 3857→CRS canônico no FastAPI.
<i>Snapping</i> via /vertices-near	—	✗	Endpoint + UI de <i>snap</i> .
Formulário de atributos validado (Zod)	Formulário de atributos (pontos)	🕒	Reaproveitar; validar contra schema do servidor.
Lock otimista + lock espacial Redis	—	✗	Versão por linha + lock em Redis.
Validação topológica (ST_IsValid/ST_Within)	—	✗	No PostGIS.
Auditoria (camadas_history, diff JSON)	—	✗	Tabelas temporais + trilha jwt . sub/timestamp/delta.
Permissão can_edit revalidada no servidor	—	✗	No PUT/DELETE.

Card 3 — Gerador de Painéis e Iframes

Requisito	GISELE hoje	Status	Ação
URL assinada (JWT com bbox/layers/exp/jti/domains)	Export de arquivo (GeoJSON/PNG)	✗	Tokenização + mini-visualizador embarcável.
Blocklist Redis (revoked:jti) + <i>kill switch</i>	—	✗	Revogação na borda + dashboard.

Requisito	GISELE hoje	Status	Ação
Cache HTTP/CDN por (jti,z,x,y)	Cache de tiles/blobs no cliente	🟡	Mover cache para a borda/CDN.
Seletor de bbox + layer picker + TTL + white-list	Seleção de área (export)	🟡	UI de empacotamento.
Marca d'água dinâmica (canvas + /preview)	—	✗	Overlay + watermark no TiTiler.
Verificação de Origin/domains	—	✗	No bundle e no gateway.

Card 4 — Portal de Importação e Aprovação

Requisito	GISELE hoje	Status	Ação
Upload resiliente (TUS/tusd)	Lê direto de FTP/arquivo local	✗	Serviço TUS → bucket de quarentena.
Fluxo assíncrono (Celery/Redis)	Helper Python síncrono	✗	Fila Celery.
Conversão GDAL → COG (/vsi3, sem download)	Decodifica no cliente	✗	Workers GDAL.
Quarentena → publicação sem mover bytes (STAC)	—	✗	Prefixos /quarantine /published + transação atômica.
Two-person rule (autor + aprovador)	—	✗	Fila de aprovação + audit_logs.
Anti-duplicação (BBox GIST + hash SHA-256)	—	✗	Índice + verificação.
Extração .zip em container descartável	Leitor ZIP no cliente (shapefile)	🟡	Mover para sandbox Docker no servidor.
Resultados de IA como GeoJSON (bbox/classe/score/XAI)	Importa GeoJSON local	🟡	Mesma esteira, sem conversão COG.

Eixos comuns (transversais)

Eixo	GISELE hoje	Ação
Back-end FastAPI gateway	Só <i>helper</i> opcional de amostragem	Construir o gateway/serviço único.
Armazenamento	Arquivos/FTP + memória	Provisionar MinIO e PostGIS; migrar dados.
MinIO/COG + PostGIS		
Entrega de tiles	Canvas próprio	Adotar TiTiler + endpoint MVT.
TiTiler/MVT		
Catálogo STAC/pgSTAC	Config manual	Adotar pgSTAC.
Segurança JWT gov.br + URLs assinadas	Inexistente	Integrar Login Único + assinatura.
Redis + Celery	Inexistente	Provisionar.
CRS sem reprojeção	Preserva origem no render	🟡 já alinhado no display; falta ST_Transform na escrita.
destrutiva		
Governança/auditoria/two-person/Meta 1	Inexistente	Trilhas de auditoria + políticas.

4. Decisão estratégica (precede tudo)

O ponto de bifurcação é o **Card 1**. Três caminhos:

(A) GISELE como front-end do Card 1, mantendo o canvas próprio. O GISELE passa a consumir TiTiler (tiles raster), o endpoint MVT (vetores) e o STAC (catálogo), atrás do gateway JWT. Reaproveita-se ~100% das ferramentas de usuário já maduras. Custo: adaptar o motor de dados; risco de divergir da spec (que pede MapLibre). *Mais rápido para um piloto.*

(B) Reescrever o Card 1 em React + MapLibre, portando a UX do GISELE. Aderência total à spec; MapLibre dá MVT/WebGL e *swipe* nativos. Custo: reescrita do front, reimplementando as ferramentas que o GISELE já tem. *Mais alinhado, mais caro.*

(C) Híbrido em fases: começar por (A) para validar o back-end com um front que já existe; migrar para (B) quando o back-end estiver estável. **Recomendado** — desrisca o back-end primeiro e preserva o GISELE como ferramenta operacional/desktop durante a transição.

Independente da escolha do Card 1, **Cards 2-4 e os eixos comuns são greenfield de servidor e seguem o mesmo backbone.**

5. Roteiro de adequação (faseado)

Fase 0 — Decisões e fundação. Escolher o caminho do Card 1 (§4). Definir CRS canônico por camada, modelo de *roles*/departamentos e o contrato do JWT gov.br. Levantar volumetria (TB de raster) para dimensionar MinIO/TiTiler.

Fase 1 — Backbone de servidor. Provisionar (Docker/K8s): **PostGIS, MinIO, Redis, FastAPI gateway, Celery**. Estabelecer o *gateway* como ponto único e o esqueleto de autenticação (validação do JWT gov.br, middleware de *role*).

Fase 2 — Entrega de dados. Subir **TiTiler** sobre COG no MinIO; implementar o **endpoint MVT** (FastAPI + PostGIS com WHERE por *claim*); publicar o **catálogo STAC/pgSTAC**. Converter um conjunto-piloto de rasters do CPTEC para COG.

Fase 3 — Segurança e assinatura. **URLs assinadas** (TTL curto) para *assets* raster; filtros de vetor por *role*; *blocklist* revoked:jti em Redis; verificação de Origin.

Fase 4 — Card 1 (adaptar o GISELE). Conforme §4: trocar o decodificador GeoTIFF local por consumo de tiles TiTiler; trocar parsers de vetor por MVT; ligar o *time-slider* e a TOC ao STAC; manter régua/área/perfil/opacidade/identificação; acrescentar *swipe*, *spatial bookmarks*, exportação PDF e o painel **XAI**.

Fase 5 — Card 2 (Editor Vetorial). API REST de geometria (POST/PUT/DELETE) com ST_Transform no servidor; /vertices-near para *snap*; lock otimista + lock espacial Redis; validação topológica; tabelas temporais (camadas_history) e can_edit. Reusar a UI de desenho do GISELE.

Fase 6 — Card 4 (Portal de Importação). **TUS** → quarentena no MinIO; **Celery + GDAL** convertendo raster→COG e validando/reprojetando vetor; publicação por troca de asset.href/status (sem mover bytes); **two-person rule**; anti-duplicação (BBox GIST + SHA-256); extração .zip em container descartável.

Fase 7 — Card 3 (Iframes/distribuição). Empacotador de recorte (bbox+layers+TTL+domínios) gerando JWT de partilha; mini-visualizador embarcável com sandbox; **marca d'água** (canvas + /preview do TiTiler); *dashboard* com **Kill Switch** e telemetria.

Fase 8 — Governança e compliance. Consolidar a **cadeia de custódia** (trilhas de auditoria em todos os módulos, *diff* JSON), a matriz de auditoria dupla e o atendimento à **Meta 1 do MPSP**; testes de penetração no fluxo de URLs assinadas e revogação.

6. O que reaproveitar do GISELE (ativos)

Vale preservar e portar: as **ferramentas de medição** (distância Haversine, área esférica, perfil de linha), a **identificação de feições por clique** (popup de atributos), os **controles de opacidade/paleta/contorno**, a **animação temporal e série temporal**, a **exportação de imagem/vídeo**, o **multipainel** (base para o *swipe*), os **parsers** (GeoTIFF/KML/Shapefile — úteis no Card 4 como referência da conversão GDAL) e a **lógica de fontes** (CPTEC/INPE/aviationweather) como conectores de ingestão. A nova **base de pontos do usuário** e a leitura de lista por arquivo já antecipam o conceito de camada vetorial editável (Card 2) e de catálogo externo.

7. Riscos e decisões em aberto

A divergência **canvas próprio × MapLibre** é o maior risco de retrabalho — decidir cedo (§4). A **volumetria de raster** define o custo de TiTiler/MinIO e a estratégia de COG/*overviews*. O **JWT gov.br** (Login Único) impõe integração e homologação específicas. O **rigor probatório do MPSP** (cadeia de custódia, *two-person*, auditoria) é requisito transversal e não pode ser adicionado “depois” — precisa entrar no desenho do back-end desde a Fase 1. Por fim, o GISELE **client-side puro não atende** sozinho a requisitos de multiusuário, segurança por *role* e auditoria: estes só existem com o back-end.

8. Síntese

A adequação não é uma refatoração do GISELE: é a **construção da plataforma de servidor COPENICUS** (FastAPI/PostGIS/MinIO/TiTiler/STAC/JWT/Redis/Celery) com o GISELE — ou sua UX — assumindo o papel do **Visualizador (Card 1)**, onde ele já é forte. O caminho de menor risco é o **híbrido (C)**: desriscar o back-end com o front existente e migrar para React+MapLibre quando estável, construindo os Cards 2, 3 e 4 como serviços novos sobre o mesmo backbone, com governança/auditoria embutidas desde o início.